

系统架构设计师

《论文写作阅评篇》



论文写作逻辑



论文框架

基本框架		内容	字数
摘要		项目相关背景及主要功能 你的岗位及主要职责 论文主体内容的总概 项目最终的实施效果或你的总结和感悟等	300字~320字
正文	(1) 项目背景介绍	项目背景的详细介绍： 项目开发的原因 项目开始时间、实施周期 你的主要岗位职责等	400字左右
	(2) 过渡内容	非核心论点问题的回应 引出主体内容（核心论点）	300~400字
	(3) 主体内容	采用总分式描述： “一总”加“三分” “一对三” 模式 可分4个段落	1000字~1500字
	(4) 论文结论	结构上可分三步走： 先分析项目运行效果 再总结项目不足 最后提解决思路	400字~600字



论文写作阅评



试题：论微服务架构及其应用

微服务提倡将单一应用程序划分成一组小的服务，服务之间互相协调、互相配合，为用户提供最终价值。每个服务运行在其独立的进程中，服务与服务间采用轻量级的通信机制互相沟通。在微服务架构中，每个服务都是一个相对独立的个体，每个服务都可以选择适合于自身的技术来实现。每个服务的部署都是独立的，这样就可以更快地对特定部分的代码进行部署。

请围绕“论微服务架构及其应用”论题，依次从以下三个方面进行论述。

- 1、概要叙述你所参与管理或开发的软件项目，以及你在其中所承担的主要工作。
- 2、简要描述微服务优点。
- 3、具体阐述如何基于微服务架构进行软件设计实现的。



论文写作阅评



2020年2月，本人所在的处室承担了业务部门提出的三方客户车贷受理系统（以下简称受理系统）的开发项目。该项目主要是由于银行业面临实体经济增速放缓、跨界融合竞争激烈、利差快速收窄，银行业经营环境面临前所未有的压力，且原有获客渠道较为单一。为了增加获客渠道，吸收更多客户，以达到银行协作风控、增长放贷总量的目标，因此受理系统立项建设。我在项目中担任技术负责人，负责项目的架构设计和技术指导工作。传统的单体架构不再适合当下互联网高并发、高可用、高性能的系统要求，而微服务架构主要有简化开发步骤、丰富的组件支持、完善的问题解决方案、易修改和扩展的优点，所以最终选用了基于Spring Cloud的微服务架构进行项目建设。本文以受理系统为例，论述微服务架构在项目中的应用。

受理系统根据业务和行内监管需要，并建设有基于ELK技术栈的日志查询体系，应用携程开源配置管理中心等。业务模块包括有批量模块、大数据模块、用户模块、进件模块、规则引擎模块、监控模块、工作流模块。均采用微服务架构风格进行开发。

一、简化开发过程，约定优于配置。

在应用的初始搭建以及开发过程，每个项目都会经历一些相同的步骤。例如服务器选型、服务器参数配置、网络配置、组件版本选择以及解决版本冲突造成的失配问题，解决这些问题往往会占用比较大的资源，影响项目进度。由于微服务采用了约定优于配置思想和实现，只需要确定Spring Cloud的主版本号，例如服务端口、服务最大网络连接数、组件版本选择等无需进行单独配置，框架会自动引入最优配置。并且这些配置已经历了无数项目的验证。简化了开发流程，使开发人员只需要关注业务进行开发，大大减少了配置文件的使用，规避了配置错误而导致的潜在风险。并且约定优于配置的思想对于修改友好，在项目实践中，如果出现配置不符合项目场景的情况下，可以自行配置参数进行覆盖。

二、丰富的组件支持

在项目建设过程中，不可避免的会用到三方组件支持。解决软件开发中通用的场景，第三方的组件能够提供更好的解决方案，在开发过程中可以使用第三方的成熟方案和实现，避免重复造轮子的情况，Spring Cloud官方有多种场景器进行支持。例如对Redis的支持，在交易中，数据查询操作往往占到交易总数的八成以上，如果每次查询都在数据库获取数据，无疑大大增加了数据库的压力，当流量波动时会影响到正常交易进行，于是项目组引入了Redis作为缓存服务器，每次进行数据查询时，先检索Redis中是否存在，当数据不存在的情况下，再去数据库中获取数据，然后把结果返回，并在Redis中存储以备下次查询，这样提高了查询速度，增强了系统性能的优点，为了达到这样的效果，项目组也仅仅是需要引入Redis的场景器即可。又如对日志体系的支持，日志记录在每个项目必须集成的一部分内容，如果缺少了日志组件的集成，则会让项目变得不可控，定位和排查问题、观察系统运行状态这些功能都需要详细和明确的日志信息，我们选择集成并使用了Spring Cloud的日志场景器，只需要对少量的必要参数进行配置，项目就可以按照要求输出日志，可以将日志输出到控制台方便调试，也可以输出到日志文件中做归档记录。再如对消息中间件RabbitMQ的支持，在业务办理完成发送短信的场景中，对于短信的发送时间和发送结果要求并不严格，因此为了保证业务主流程能够正常继续进行，使用异步发送短信，具体为受理系统将短信内容、手机号、客户姓名等信息发送至RabbitMQ的指定队列，并由短信系统监听队列，对消息进行消费并发送短信。项目组只需要引入Spring Cloud支持的消息中间件场景器就可以做到。



论文写作阅评



三、完善的问题解决方案

对于系统建设中的场景几乎都提供了解决方案供参考。应用网关应当是外部系统访问系统的唯一入口，网关应该具有请求转发、地址映射、路由配置、鉴权控制等功能。在Spring Cloud中提供了多种网关的解决方案供选择，如典型的Zuul和Gateway都能很好的完成以上功能要求。网关对于访问请求地址的解析，可以将请求分别指派到受理系统的相应模块上，并且可以起到对外隐藏了真实地址的作用。最终由于Gateway有更好的扩展性，受理系统选用了Gateway作为服务网关。服务注册中心最主要功能有服务注册与发现，服务启动时可以根据服务名称将IP地址和端口等信息注册到服务注册中心，服务注册中心心跳机制定期检测服务是否存活。如果存在服务宕机的情况，注册中心会在注册表中删除该服务，直到该服务再次启动注册。由于服务会定期到注册中心获取最新的服务列表，这样在服务之间相互调用时，会调用到正常运行的服务，避免系统出现长时间的业务报错。Spring Cloud提供了Zookeeper和Eureka的支持，由于Eureka具有良好的页面支持查询，最终受理系统选择了Eureka作为注册中心。由于大部分服务都是集群部署，即同一个服务会部署多个实例，请求的分发策略会影响到系统的吞吐量和服务器的利用率，因此负载均衡成为了每个系统必须要解决的问题。Spring Cloud同样给出了多种解决方案。基于客户端的负载均衡策略，例如Ribbon和RestTemplate可以对服务按照轮询、加权轮询、随机、一致性Hash等策略分发请求。在受理系统的实践中，发现Feign具有良好的性能和简洁的使用方法，最终选择了Ribbon作为负载均衡组件使用。

三、易修改行

由于微服务按照领域模型进行业务拆分，使服务之间松散耦合，代码不再臃肿堆叠，在每个服务内只关心领域内的变更。因此在开发新的业务需求时，不需要对项目整体进行升级，只需要对需求变更涉及到的微服务进行升级即可。这样的做法避免了可能存在需求之间的冲突。同样由于业务领域的拆分，在测试阶段也不再需要进行全部功能回归测试，只需要对少量服务进行针对测试即可。极大的简化了开发流程和测试流程，缩短了需求开发到上线的时间。

我们在最初设计的时候，也存在一些不足。未考虑到复杂业务场景中 workflow 系统的必要性。在时间中发现即使与外部合作方数据交互流程大致相同的情况下，但由于业务需要银行内部对数据处理的流程却各不相同，导致已有功能无法达到复用的效果，大大增加了研发的工作量。对于这样的情况引入了 workflow 引擎帮助我们去实现非常灵活的流程设计，并对 workflow 引擎进行二次开发，使 workflow 引擎更易使用，加快了开发效率，提高开发质量，极大的实现了数据流转的规范化。在项目组应用后解决了这个问题，并在后续的多轮迭代中得到了验证，提高了在同行业中的竞争力和影响力。



论文写作阅评



2021年4月，我司承接了电信某省公司的数据安全管控平台的项目，该系统主要实现数据源自动识别、敏感数据、重要数据分类分级，接口自动发现、接口阻断、接口脱敏、接口脆弱性评估、接口敏感数据发现、告警通知、报表定制化等功能模块。我在该项目中担任系统架构师，主要负责该项目的技术方案制定及技术指导。本文以数据安全管控平台为例，主要论述微服务架构在该项目中的具体应用，首先技术选型上，综合需求层面及团队技术基础考虑，确定了spring cloud alibaba作为该项目的微服务框架。其次通过组合各个服务组件（gateway、nacos等），快速进行业务服务的开发。最后针对接口镜像流量大、处理速度慢的问题，进行了一系列性能优化，通过以上技术的应用，项目最终顺利上线，达到了客户的期望，获得客户的一致好评。

随着《数据安全法》、《个人信息保护法》的相继出台，各个行业都亟需一套数据安全管控平台来满足国家对数据安全以及个人信息保护的要求，尤其是在国民经济中起着重要作用的运营商。

2021年4月我司承接了电信某省公司的数据安全管控平台的建设。数据安全管控平台包括数据源的自动识别，敏感数据、重要数据的分类分级，接口自动发现、接口阻断、接口脱敏、接口脆弱性评估、接口敏感数据识别，告警通知、报表定制化功能模块。主要实现对手动配置或自动扫描到的数据库资产的定时识别检测，识别出被扫数据库中的敏感数据，如（用户名、手机号、密码等），根据内置的配置数据对这些敏感数据、重要数据分类分级、并打上标签，存储到识别结果中。接口安全通过采集网络中的镜像流量、自动发现接口、识别接口中的敏感数据，发现接口的脆弱性，接口安全通过串行部署还能对接口进行脱敏、阻断等更安全接口的管控。当数据安全管控中心检测到安全威胁时可以通过告警中心向相关人员发出告警。报表中心用来定制对外展示的不同维度的报表。我在该项目中担任系统架构，主要负责系统架构设计和技术指导工作。最终该项目在2021年12月顺利上线。

微服务是一种通过多个小型服务的组合，来构建单个应用的机构风格，有如下几个特点：1.围绕业务能力而不是特定的技术标准来构建服务，各个服务可以是采用不同的编程语言、不同的数据存储技术运行在不同的进程之中。2.分散治理，微服务对应的开发团队、有着直接对服务运行质量负责的责任。3.通过服务来实现独立的组件。4.数据去中心化，微服务这种架构也明确地提倡数据应该按领域来分散管理、更新、维护和存储。5.轻量级的通信机制、比如说restful风格的通信。6.容错性设计，在微服务的设计中，有自动的机制能够对其依赖的服务进行快速故障检测，在持续出错的时候进行隔离，在服务恢复的时候重新联通，可靠系统完全可以由会出错的服务来组成，这是微服务的最大价值所在。

在实际的项目实践中，我们选择了spring cloud alibaba来做我们的微服务开发框架。下面着重从框架的选择、框架的应用以及性能调优等三个方面来具体阐述。

框架选型

从数据安全管控平台的整体层次上来讲，为了实现平台的开发性和扩展性，我采用了平台加能力的模型来实现业务需求。告警中心、报表中心等通过系统服务来实现。数据识别服务提供数据库敏感数据识别以及分类分级。接口安全服务提供接口数据的识别、脱敏、脆弱性评估。我们为这些服务选择了spring cloud alibaba这套开发框架来实现我们的服务。spring cloud alibaba是spring cloud的子项目，服务spring cloud的标准。nacos discovery 和eureka性能强劲、感知更快，Gateway 性能是zuul的1.6倍，nacos config 和spring cloud config比，搭建简单，有可视化界面，配置管理更高效，学习曲线低。因此，从两种框架的各种组件对比来看，spring cloud alibaba 更适合我们的需求和更受我们开发团队的认可。



论文写作阅评



框架应用

数据安全管控平台由系统服务、接口安全服务、数据识别服务、网关服务，我们平台的认证功能，黑白名单功能、限流功能、负载均衡功能都放到了网关服务来实现，用户登录成功后token被存储在redis中，所有需要认证的接口请求通过网关的时候，首先进入认证过滤器，过滤器先从请求header中获取token，然后去redis中查找该token是否存在，存在就往后转发，不存在就拒绝该请求。redis的引入提高认证的性能又能解决了分布式下token一致性的问题。黑白名单配置在网关的配置文件中，认证过滤器认证前先判断请求是否在白名单中，如果在就绕过认证校验，不在就进入认证环节。如果请求链接在黑名单中，就直接拒绝该请求。微服务间的接口调用，通过openfeign来实现，每个需要对外提供接口的服务，通过@FeignClient注解定义对外接口，并把这些接口文件放到单独的包里面，其它服务通过pom文件引入这个包后，就可以像调用本地方法一样调用远程服务的方法了。服务间接口调用的时候通过统一的拦截器把原始请求头里面重要信息传递到下一个请求的header里面，如token。

性能调优

数据安全管控平台的性能瓶颈主要主要体现在接口流量识别和处理上。为此我们先把接口的镜像流量采集到kafka，然后通过接口服务消费kafka里面对应topic的接口流量数据进行处理，再把处理后的数据存储在elasticsearch集群里。按照这套流程处理下来，数据识别接口的处理速度1分钟6000次接口流量。这个速度远不能满足客户的业务需求，如是我们从一下三个方面对系统进行了优化：1.调整kafka消费者的参数,开启kafka的批量消费功能。2.对接口服务的消费处理逻辑的线程池的核心线程数、最大线程数以及等待队列的大小。3.调整elasticsearch批量入库的参数，批量入库的时间间隔、批量入库的数据量、批量入库的数据大小，上面的三个参数有一个满足的时候就会触发一次入库动作。通过对上面三个方面参数综合调整优化后，在相同的硬件资源下，数据处理速度达到了每分钟60000次。后面根据接口镜像流量的大小，还可以通过增加kafka的partition的数量和接口流量消费服务的数量来对系统进行动态扩容，从而满足更大流量处理能力。

通过以上技术应用，项目经历8个月周期，数据安全管控平台最终于2021年12月顺利上线，目前系统运行正常，无论是业务系统的稳定性还是数据处理的速度都达到了预期，获得了客户各级管理层的一致好评。

但是在该系统的实施过程中，其实也发现了微服务的一些不足之处，首先对应的服务开发人员都熟悉自己开发的服务，对其它服务却知之甚少。其次就是服务众多，出现问题和异常时，排查起来也比较困难，没有单体服务那么容易了，对开发者的整体要求也高了。当然，对于第一问题我们要求开发人员不仅要熟悉自己开发的服务，对其它的服务也要有所了解，便于整体理解系统。对于第二个问题我们可以采用skywalking这个组件来帮助我们完成调用链路分析以及应用监控统，skywalking支持多张插件、UI功能较强，接入端无代码侵入,而且性能在众多的竞品中也是佼佼者。



论文写作分析



试题：论基于构件的软件开发

软件系统的复杂性不断增长、软件人员的频繁流动和软件行业的激烈竞争迫使软件企业提高软件质量、积累和固化知识财富，并尽可能地缩短软件产品的开发周期。集软件复用、分布式对象计算、企业级应用开发等技术为一体的“基于构件的软件开发”应运而生，这种技术以软件架构为组装蓝图，以可复用软件构件为组装模块，支持组装式软件的复用，大大提高了软件生产效率和软件质量。

请围绕“基于构件的软件开发”论题，依次从以下三个方面进行论述。

- 1.简述你所参与开发的运用了构件技术的项目，以及你所担任的工作；
- 2.论述你在项目中如何运用构件技术来进行软件开发；
- 3.分析并讨论各种构件技术的优点、缺点，并展望构件技术的发展趋势。

2022年4月，我公司承接了一项某省营商环境监测服务平台应用及支撑系统建设项目，包括营商环境综合服务门户、数据采集系统、动态监测分析系统、可视化系统的应用体系和营商环境数据管理系统、数据服务系统、用户管理对接系统的应用支撑体系。在项目中我担任系统架构师，主要负责架构设计工作。本文以该平台为例，论述了基于构件的软件开发技术在本项目中的具体应用。经过需求分析与构件规划、构件库建设与选择、接口标准化与设计、构件开发与测试、支撑系统构建、可视化与监控、维护与优化等步骤和策略，有效地确保不同构件在营商环境监测服务平台中高效、有序地集成，形成一个既灵活又稳定的系统。最终项目顺利上线并稳定运行，获得用户一致好评。

近年来，党中央、国务院高度重视深化“放管服”改革、优化营商环境工作，部署出台了一系列有针对性的政策措施，优化营商环境工作取得积极成效。但同时我国营商环境仍存在一些短板和突出问题，我省已连续三年进行了省域范围内的营商环境第三方评价工作，但是评价数据来源目前还是以政府填报、企业填报的形式为主，营商环境监测服务平台的建设，将通过系统化、信息化的方式，多渠道汇集营商环境相关的业务数据、评价数据等，并通过数据治理、数据清洗加工等确保数据的规范性、完整性、准确性以及及时性。在这个项目中，我担任了系统架构师的职务，主要负责系统的架构设计相关工作。通过组合现有的、可重用的软件部件来快速开发高质量的软件系统，从而提高开发效率、减少错误、缩短开发周期，常用的构件技术有三种：CORBA、EJB和COM，CORBA是一种跨语言、跨平台的分布式对象模型,提供了高度的灵活性和强大功能，但也相对复杂，配置和管理成本较高。EJB容器负责管理事务、安全、持久化等服务，减轻了开发者负担,但是应用起来复杂，过度重量级，消耗资源。COM易于集成和使用，支持多种编程语言，但主要与Visual C++、VB等微软技术栈紧密集成。

采用服务注册中心来管理构件的注册与发现过程，新加入或更新的构件可以自动被系统识别和调用，提高了系统的动态性和扩展性。利用配置管理工具来集中管理不同构件的配置信息，确保不同环境下的配置一致性，便于构件的灵活部署和配置更新。采用Docker容器技术封装构件及其依赖，确保构件在不同环境下的运行一致性。结合容器编排工具来自动化部署、扩展和管理容器化的构件，提高部署效率和资源利用率。建立自动化构建、测试和部署流程，确保每次代码变更后，所有相关的构件都能够被重新构建、测试并安全地部署到生产环境，保证集成的及时性和质量。模块化设计与解耦，每个构件内部高内聚、低耦合，通过设计清晰的模块边界和抽象的接口，减少构件之间的直接依赖，便于单独开发、测试和替换。使用版本控制系统管理代码版本，确保不同构件间的依赖关系得到妥善管理，避免版本冲突。本文重点从构件标识、构件库的建设与选择、构件开发与测试三个方面论述了营商环境监测服务平台应用及支撑系统采用构件化开发的过程。

1、构件标识

对平台的整体需求进行深入分析，明确平台的各个功能模块，识别出营商环境综合服务门户里工作台模块、内容管理模块、栏目管理模块、配置模块、网站统计模块、互动模块、会员模块、系统模块，营商环境数据采集系统互联网数据采集系统里功能模块、问卷信息采集系统里的功能模块，营商环境动态监测分析系统里指数分析功能模块、绩效考核系统功能模块、分析报告系统里功能模块，根据功能点的逻辑关系和相互依赖性，将它们组织成逻辑上独立、功能上完整且尽可能通用的模块。每个模块具备明确的职责和边界。为每个模块定义清晰的服务接口，包括输入输出参数、数据格式、调用协议等，确保模块间的松耦合和高内聚。利用接口定义语言（如OpenAPI）来标准化接口描述，便于跨团队沟通和集成。识别出可重用的技术组件，数据处理组件、用户认证模块、权限管理组件、消息队列服务等，这些与具体业务逻辑分离，具备通用性。对于业务逻辑复杂的功能，考虑将其划分为多个业务组件，如数据清洗组件、指标计算组件、预警规则引擎等，每个组件专注处理特定的业务逻辑。评估每个候选组件的复用潜力，优先考虑那些可以在多个应用场景中复用的组件。确保组件设计遵循业内标准或最佳实践，如遵循微服务架构原则，确保组件的通用性和可替换性。在识别构件时，考虑长远的维护和扩展性，避免设计出难以维护或不易扩展的组件。为每个识别出的构件编写详细的文档，包括设计目的、接口说明、使用示例、依赖关系等，便于团队成员理解与使用。



论文写作阅评



2、构件库的建设与选择

项目组首先确定了构件的技术栈（Java），制定编码规范、接口协议标准和数据交换格式（XML）。使用版本控制系统管理构件的源代码，同时建立构件的文档系统，详细记录构件的功能、使用方法、依赖关系等。为每个构件编写单元测试、集成测试脚本，确保构件的健壮性。采用代码审查、静态代码分析工具加强质量控制。搭建或使用现有的构件注册中心，用于构件的存储、版本管理、依赖解析。评估项目团队已有的构件资源，再考察外部市场上的成熟构件。对外部构件进行兼容性测试，确保与现有系统和技术栈的无缝集成，同时进行性能评估，确保满足项目需求。检查外部构件的安全性，避免引入潜在的安全漏洞。同时，确保遵守其许可证协议，避免法律风险。选择活跃度高、文档齐全、社区支持良好的外部构件，有助于解决集成过程中遇到的问题。考虑构件的长期维护和升级路径，优先选择有良好维护记录和明确升级计划的构件。有效地建设和管理构件库，不仅提升开发效率，也能确保项目质量和可持续发展性。

3、构件开发和测试

基于业务需求我们选择了合适的设计模式，工厂模式、单例模式和微服务架构，确保构件的高内聚低耦合。遵循预定义的编码规范，使用选定的编程语言和框架进行开发。重视代码的可读性和可维护性。为每个构件编写单元测试，覆盖主要逻辑路径和边界条件。使用JUnit、pytest等单元测试框架，确保代码质量。同步编写技术文档，包括构件的功能说明、使用方法、配置选项、依赖关系等，便于团队成员理解和使用。将构件集成到系统中，验证构件间交互是否符合预期，对关键构件进行压力测试和负载测试，确保在高并发场景下系统稳定运行。同时进行渗透测试、代码审计，检测潜在的安全漏洞，确保数据安全和隐私保护。对于前端构件，实施用户体验测试，确保界面友好、操作流畅，并能收集用户反馈。设置自动化测试和部署流程，每次代码提交后自动执行构建、测试和部署，加速迭代速度。每次更新或修复后，执行回归测试，确保原有功能未受影响，维护软件质量的稳定性。

经过近10个月发开发和部署，2023年2月份平台顺利投入使用，以可视化大屏的方式展示各城市营商环境指数排名，展示营商环境指标体系相关的内容，展示营商环境绩效考核结果，展示市场主体投诉举报结果等。平台自正式上线已运行一年有余，获得了使用部门的一致好评。随着微服务、云原生平台的兴起，同时低代码和无代码开发平台的兴起，使得非专业开发者也能通过拖拽式构件快速构建应用，这推动了构件化技术向更广泛的用户群体普及，加速数字化转型。基于构件技术的发展会向更程度的自动化、智能化、标准化和灵活性上迈进，这个过程离不开我们每一个软件从业人员的努力奋斗。



论文写作阅评



本文以在线课程学习平台为例，主要论述了基于构建的软件开发在该项目中的具体应用。2019年3月，我司自主开发了某在线学习平台，该平台集成了门户、学习中心、教学管理中心、社交系统、系统管理等功能模块。我在该项目中担任系统架构师的角色，主要负责系统整体的架构设计工作。本文以该平台为例，论述了基于构件的软件开发技术在本项目中的具体应用。系统基于Spring Cloud微服务框架来进行构件化开发，在构件获取阶段通过对接现有系统、使用构件库、集成第三方软件来实现需求，在构件开发阶段使用了多种设计模式来保证构件的可重用性，在构件组装阶段使用了同步消息、异步消息、工作流方式来实现不同业务类型下构件的组合。最终项目顺利上线并稳定运行，获得用户一致好评。

受互联网+概念的催化，当今中国在线教育市场的发展可谓是百花齐放、如火如荼。按照市场领域细分为：学前教育、K12教育、高等教育、留学教育、职业教育、语言教育、兴趣教育以及综合平台，其中，职业教育和语言教育的市场优势突出。根据Analysys易观发布的数据显示，预计2020年中国互联网教育市场交易规模将达到3718亿元人民币，未来三年互联网教育市场规模保持高速增长。

2019年3月，我所在公司着手开发一款在线学习的平台系统，该系统借鉴了MOOC（大型开放式网络课程，即MOOC（massive open online courses））的设计思想，是一个提供IT职业课程在线学习的平台。该系统为即将和已经加入IT领域的技术人才提供在线学习服务，主要包括门户、学习中心、教学管理中、社交系统、系统管理等功能模块。门户是整个平台的入口，功能包括：门户首页、注册/登录、课程搜索、职业规划，客服等。学习中心为用户提供在线学习服务，包括：我的课程、视频点播、视频直播、在线考试、在线答疑、学习统计等功能；教学管理中心为教育机构或个人讲师提供教学管理功能，包括：课程管理、媒资管理、考试管理、问答管理等功能；社交系统模块为老师和学生交流搭建沟通的平台，包括：问答系统、评论系统、论坛等；系统管理中心提供系统参数配置、CMS、数据字典、分类管理等功能。在这个项目中，我担任了系统架构师的职务，主要负责系统的架构设计相关工作。

构件技术是指通过组装一系列可复用的软件构件来构造软件系统的软件技术。通过运用构件技术，开发人员可以有效地进行软件复用，减少重复开发，缩短开发时间，降低软件的开发成本。主流的构件技术有三种：CORBA、EJB和COM。CORBA分为对象请求代理、公共对象服务和公共设施三个层次，优点是大而全，互操作性和开放性好，缺点是庞大且复杂，技术标准更新缓慢；EJB基于Java语言，支持跨平台，提供了远程访问、安全、持久化和生命周期等机制，支持分布式计算，缺点是服务治理能力差，逐渐被Spring Cloud等轻量级框架取代；COM基于Windows平台，功能强大、效率高，有一系列开发工具支持，缺点是跨平台性差。基于构件的软件开发过程主要分为模块划分、构件标识、构件获取、构件组装与测试、构件管理等步骤。

在线学习平台采用前后端分离的技术架构，前端采用vue.js技术栈，服务端采用SpringBoot、SpringCloud等Spring全家桶技术栈。。这里重点从构件获取、构件开发、构件组装三个阶段说明本系统采用构件化开发的过程。



论文写作阅评



1. 构件获取

在线学习平台中，可复用的功能需求和非功能需求从实现方式上分为三类。第一类需要对接现有系统实现，比如平台登录入口采用多种登录方式和微信，QQ，微博联系开发商，取得了相关接口调用的SDK，同时还有短信服务，收件服务，点播直播，CDN服务等也都是和相应的开发生做了对接。第二类是常见信息系统中共同具备的用户管理、角色权限管理、日志记录、内容维护、消息中心等基本功能，我们通过取用单位过往项目开发中所积累的构件库中的相应构件（例如RBAC权限管理框架）来实现。第三类需要集成第三方的软件来实现，例如微服务注册中心、消息中间件，网关、负载均衡机制，缓存，分布式文件系统，无需另行开发，我们使用了Spring Cloud中的Eureka框架来作为微服务注册中心，Nginx反向代理来作为负载均衡机制，redis作缓存，HLS技术实现视频点播功能，Ribbon作为微服务客户端进行远程调用，针对大文件上传功能，我们使用了GitHub上的开源程序。

2. 构件开发

构件的优势体现在其粗粒度的重用性，因此为最大限度保证构件的可重用性和重用力度，保持其高内聚、低耦合的特性，以捕获异常为例，我们对于系统对异常的处理使用统一的异常处理流程，对于不可预知的异常（运行时异常）由SpringMVC统一捕获Exception类型的异常，由统一的异常捕获类来解析处理，并转换为与自定义异常类型一致的信息格式（错误代码+错误信息）。可预知的异常及不可预知的运行时异常最终会采用统一的信息格式（错误代码+错误信息）来表示，最终也会随请求响应给客户端。此外对于数据的返回格式，我们也做了统一的封装，使得前端处理数据的逻辑统一。

3. 构件组装

在线学习平台中不同的业务类型，需要采用不同的构件组装方式。在本系统的开发过程中，我们用到主要应用了以下几个方面。课程搜索方面采用ElasticSearch+LogStash组合完成，使用Logstash（logstash是ES下的一款开源软件，它能够同时从多个来源采集数据、转换数据）将MySQL中的课程信息读取到ES中创建索引，使用IK分词器进行分词。在多端口登录认证方面采用认证服务+Redis+Zuul方式，实现用户的登录认证和系统间的权限共享。在订单支付方面，采用异步消息的方式，用户创建订单，通过消息中间件异步通知课程管理，增加课程资源，并返回结果通知，同时采用定时任务对创建失败的订单进行重试。

系统自2020年5月正式上线半年内，注册用户达百万级，并且还在持续增长，得用用户和机构的肯定。

实践证明，在线学习平台能够顺利上线，并且稳定运行，与系统采用了基于构件的开发方式密不可分。经过这次构件化开发的方法和实施的效果后，我体会到了软件元素重用对开发过程的重要价值。从软件开发的方式看，由机器语言、汇编语言，到面向过程开发、面向对象开发，再到现在基于构件、面向服务的软件开发，软件元素在两个维度上呈现进化趋势：内部功能越来越强大、全面，对外的接口越来越简单、标准。最终各领域软件可在一个统一标准下无缝组装，届时面向协作的软件开发、基于职能的软件开发等新技术都将出现，上层应用功能的实现也将变得异常简单，计算机软件将会无所不在，数字化生活、智能地球等现在还处于概念阶段的事物，将会走进现实。这个目标值得我们每一位软件从业人员为之努力奋斗。



论文写作分析



试题：论高可靠性系统中软件容错技术的应用

容错技术是当前计算机领域研究的热点之一，是提高整个系统可靠性的有效途径，许多重要行业（如航空、航天、电力、银行等）对计算机系统提出了高可靠、高可用、高安全的要求，用于保障系统的连续工作，当硬件或软件发生故障后，计算机系统能快速完成故障的定位与处理，确保系统正常工作。

对于可靠性要求高的系统，在系统设计中应充分考虑系统的容错能力，通常，在硬件配置上，采用了冗余备份的方法，以便在资源上保证系统的可靠性。在软件设计上，主要考虑对错误（故障）的过滤、定位和处理，软件的容错算法是软件系统需要解决的关键技术，也是充分发挥硬件资源效率，提高系统可靠性的关键。

请围绕“高可靠性系统中软件容错技术的应用”论题，依次从以下三个方面进行论述。

1. 简述你参与设计和开发的、与容错相关的软件项目以及你所承担的主要工作。
2. 具体论述你在设计软件时，如何考虑容错问题，采用了哪几种容错技术和方法。
3. 分析你所采用的容错方法是否达到系统的可靠性和实时性要求。



论文写作阅评



2021年9月，我单位联合某地区团委研发了青燕归巢应用平台。该项目主要是帮助高校和企业共同助力大学生实习，实现大学生从在校实践到毕业工作的全链条服务。我在该项目中担任系统架构师。主要负责架构设计工作。本文以该平台为例，主要论述软件容错和方法在该系统中的具体应用。服务器通过采用集群化模式部署；数据库通过主备模式进行部署；软件程序通过springcloud全家桶中的Hystrix来进行服务的降级熔断。事实证明，以上软件的容错技术的应用对于系统的安全性，可靠性都起了很好的作用，满足了系统的性能需求，并且上线到现在一直稳定运行，并扛住了暑期前高并发的应用场景，得到了领导和同事的认可和赞赏。

2021年9月，我单位和某地区团委聚焦大学生青年群体就业实习实践难、企业招工难、城市留人难等问题，联合开发“青燕归巢”数字化应用平台。建设区域“人才库”和“岗位库”，实现人才和岗位的精准匹配。该平台主要有：用人单位发布岗位，青年报名、审核录用，办理入岗，获取实践报告等功能模块。我负责该平台的架构设计师，主要负责该平台的设计工作。

平台采用微服务的架构风格，后端采用java为核心开发语言，将平台服务划分为三类服务，核心服务，pc/移动端web服务，平台保障服务。其中核心服务主要包括岗位库服务，报名审批服务，人才库服务，，其他政策性服务。平台web服务主要提供与用户对接的交互界面，平台保障服务主要包括例如JOB框架，平台服务通知，MQ等服务。对于平台中的一些热门岗位，因为实践人数限制，不可避免的遇到大量的网络请求，对于软件的高平发，高可用性有很高的要求，所以该平台的核心服务必须具备一定的容错能力，在高平发的场景下，用户使用起来依旧丝滑，不影响其他服务的正常使用，这就要求在软件架构设计中必须考虑到软件容错技术的应用。

提高软件系统可靠性的技术主要分为：容错技术、避错技术。容错技术主要通过冗余实现，分为结构冗余、时间冗余、信息冗余、冗余附加。结构冗余又分为静态冗余、动态冗余和混合冗余。软件容错技术主要有N版本程序设计、恢复块方法和防卫式程序设计。结合Web软件的性质，我主要采用了集群技术、数据库主从方式和程序设计方面来进行软件的容错与避错处理。下面就从以上三方面详细讨论我所采用的容错技术和方法。

1. 集群技术

平台中各服务如果在运行时部署在一台服务器上，那么很容易出现单点故障，导致整个项目的不可用，所以一般中大型的项目，都会采用集群部署。该平台使用Nacos作为服务中心，实现服务间的注册及发现，提供了对服务的实时健康检查，阻止了web端向不健康的主机或服务发送请求，在结合openFeign负载均衡策略进行分流。例如当一个用户点击报名按钮，则首先通过网关找到在注册中心找到对应的服务，如果有一台机器宕机，不可用了，则因为Nacos的心跳健康检查，会将该服务维护为不健康服务，后是直接下线。如果服务是健康状态，在会根据我们配置的负载均衡策略：最小连接法，会将请求平滑分配给最小连接数的后端服务，这样的架构以简单的方式扩展了服务器和网络的带宽，可以大大提升系统的并发量，同时保证了后端服务的高可用性。



论文写作阅评



2. 数据库主从部署

对于热门岗位，或者是相同地区有限的岗位，都会成为青年人眼中的“香饽饽”，就会导致大量的请求，还是会落在同一个岗位身上，因为我们采用的是分布式的微服务架构，单体服务间的锁不起作用，就好比秒杀场景，会导致数据库层面瞬间收到大量的网络请求，导致数据库层面资源被占用，从而大量请求都在等待数据库响应，拖垮服务的现状，所以首先该平台采用了主从部署，实现了读写分离。因为岗位在线状态是不可修改，所以分库可以帮主库承担一部分读的流量。并且考虑到磁盘的IO性能瓶颈，我们还采用了redis作用我们的中心缓存，将岗位提前部署于redis缓存之中，并且redis也采用了主从部署，保证大量请求可以被redis吸收，因为redis是基于内存读写，大大提高了系统响应，减少了对数据库层面的请求，保证了数据库的高容错性

3. 程序设计方面

由于外部其他服务是否健康的不可控的因素，还是会有大量的请求通过客户端涌下服务端，然而客户端服务因为各种原因可能会出现等待时间超长，大量请求流量堆积，所以本平台还引用了Hystrix，通过延迟容忍和容错逻辑，控制分布式服务之间的交互。它通过隔离服务间的访问点、防止级联失败和提供回退选项，保证系统的整体弹性，当调用等待时间过长，或者线程池满了，直接进行熔断，返回给客户端错误信息，例如：该岗位过于火热，请稍后再试。避免全部线程阻塞，导致服务器雪崩的发生，大大保证各个服务的健康，整个平台的稳定运行。

系统自2022年5月份运行以来，共发布各类岗位1w余个，累计注册用户数2w多人，报名1.7w人次，并且顺利度过暑期高峰期。获得领导，同事的一致好评。

在试运行期发现了redis的主从模式不具备自动容错和恢复功能，主节点故障，集群无法进行工作，可用性低，从节点升级主节点需要人工干预，从而最后使用了最为性价比的哨兵模式，哨兵直接会相互监控，会通过竞争机制从节点中竞选出新的主节点，保证了redis集群的更高的容错性。

实践证明，该项目能够顺利上线，并且稳定运行，与系统的可靠性设计密不可分。经过这次软件容错技术的方法和实施的效果后，我也看到了自己的身上的不足，在未来还会不断地更新知识，完善本系统在可靠性方面的设计，使整个系统能够更加好用，更有效地服务于社会。



论文写作阅评



试题：论软件系统架构风格

系统架构风格 (System Architecture Style) 是描述某一特定应用领域中系统组织方式的惯用模式。架构风格定义了一个词汇表和一组约束，词汇表中包含一些构件和连接件类型，而这组约束指出系统是如何将这些构件和连接件组合起来的。软件系统架构风格反映了领域中众多软件系统所共有的结构和语义特性，并指导如何将各个模块和子系统有效地组织成一个完整的系统。软件系统架构风格的共有部分可以使得不同系统共享同一个实现代码，系统能够按照常用的、规范化的方式来组织，便于不同设计者很容易地理解系统架构。

请以“软件系统架构风格”论题，依次从以下三个方面进行论述：

1. 概要叙述你参与分析和开发的软件系统开发项目以及你所担任的主要工作。
2. 分析软件系统开发中常用的软件系统架构风格有哪些？详细阐述每种风格的具体含义。
3. 详细说明在你所参与的软件系统开发项目中，采用了哪种软件系统架构风格，具体实施效果如何。



论文写作阅评



本人于2020年5月参与公司与某集团合作的“牛仔布生产管理RFID”项目，该系统为牛仔布原料追踪，生产流程调优、仓储物流计划，数据可视化等方面提供全方位的软件支撑，在该项目组中我担任系统架构师岗位，主要负责整体架构设计与中间件选型。本文以该RFID项目为例，主要讨论了软件架构风格在该项目中的具体应用。底层架构风格我们采用了虚拟机风格中的解释器。因该牛仔布织厂对应很多合作的染厂都有各自的数据格式，比如相同一个色号有多种表示方式。使用解释器风格可以满足数据格式兼容性需求；中间层关于应用层的数据流转我们采用了独立构件风格中的隐式调用，这种风格主要用于减低系统间耦合度、简化软件架构，提高可修改性方面的架构属性；应用系统层我们采用了B/S的架构风格，统一解决传统生产行业性难题“原料浪费、数据不准确、仓库位置浪费”等问题。最终项目成功上线，获得用户一致好评。

随着某集团董事会对牛仔布生产信息化管理重视程度越来越高，该集团自2020年1月起与我司合作，规划到2021年厂区各个生产流程受RFID读写器天线监控的比例达到80%以上，同时配套将RFID生产管理系统建设被列为工作重点。不管从信息化生产管理补贴监管、安全监控或者公交公司自身的营运和机护需求，都要求有RFID生产系统对他们进行全方位的支持，而我司是该公交的主要RFID读写器与天线、EPC标签、EPC打印机产品的主要供应商，我司不仅熟悉掌握仓储生产系统WMS、MES的开发，而且有RFID硬件与软件集成非常明显的领域知识优势，因此2020年1月我司被该集团委托建设牛仔布生产管理RFID项目。本项目组全体成员共有10人，我在项目中为担任系统架构师职务，架构小组共2人，我主要职责负责整体架构设计与中间件选型，4月份完成架构工作，整个项目共耗时了7个月，2020年12月顺利通过验收。

在架构工作开始阶段，我们便意识到，架构风格是一组设计原则，是能够提供抽象框架模式，可以为我们的项目提供通用解决方案的，这种能够极大提高软件设计的重用的方法加快我们的建设进程，因此在我司项目经理的建议下，我们使用了虚拟机风格、独立构件风格以及B/S架构风格这三种较常用风格。虚拟机风格中的解释器架构风格能够提供灵活的解析引擎，这类风格非常适用于复杂流程的处理。独立构件风格包括进程通讯风格与隐式调用风格，我们为了简化架构复杂度采用了隐式调用风格，通过消息订阅和发布控制系统间信息交互，不仅能减低系统耦合度，而且还提高架构的可修改性。B/S架构风格是基于浏览器和服务器的软件架构，它主要使用http协议进行通信和交互，简化客户端的工作，最终减低了系统维护的难度，以下正文将重点描述架构风格的实施过程和效果。

底层架构我们使用解释器风格来满足不同染厂色号数据格式不一致的需求。解释器风格是虚拟机风格中的一种，具备良好的灵活性，在本项目中我们的架构设计需要兼容好10种不同染厂数据格式，一般来说这种软件编写难度非常高，代码维护难度压力也很大，因此这个解释器的设计任务便很明确了，软件设计需要高度抽象、协议的适配由配置文件来承担。具体的做法如下，我们对各染厂的色号数据结构进行了高度抽象，由于色号数据由多个部分组成，每个部分固定并且标识和数据有明确规定，因此我们将色号数据格式进行关系建模，使不同色号数据结构映射成RGB标准的色系数据结构，核心的代码采用jdom.jar与java的反射机制动态生成java对象，搭建一套可以基于可变模板的解释器，协议模板的产生可以由公交公司提供的excel协议文档进行转换得到，解释器支持协议模板热部署，这种可以将透传二进制数据直接映射成java的可序列化对象，将数据协议的复杂度简化，后期数据协议更改不会对软件产生影响，仅仅更改协议模板文件即可，最终我们使用了10个协议描述文件便兼容了这些复杂的色号数据格式，规避了色号格式巨大差异带来的技术风险。



论文写作阅评



中间层我们使用独立构件风格中的隐式调用来简化构件间的交互复杂度，降低系统耦合度。主要的实现手段是，我们采用了一个开源的消息中间件作为连接构件，这个构件是apache基金会下的核心开源项目activemq，它是一款消息服务器，其性能和稳定性久经考验。经过activemq分发到全厂与系统交互的RFID读写器，这些控制命令包括重启读写器，调整读写器连接的天线功率等，由于同时发送操作命令给相同的RFID读写器会造成读写器只执行其中一条的问题，而这种以队列管理消息发布与消息订阅的机制，能够有效解决耦合问题，错误率降低，而且我们使用的spring框架，消息队列的管理完全基于配置，清晰简单，维护性良好，例如读写器控制、EPC信息上传、EPC打印机控制等消息队列分类清晰，可以随时修改其结构也能够随时增其他主题的消息队列，不同的web系统监听的队列也可以随时变换组合，基于消息中间件的架构设计能够让系统的构件化思路得到良好实施，总体来说这种架构风格带来了非常清晰的数据流转架构，简化了编码难度，减低本项目的二次开发的难度。

应用系统层我们主要采用B/S的架构风格，主要用于解决系统更新与维护难得问题。RFID系统特点，每个机台旁边都需要开启对应数据显示屏，整个厂区起码有上百台显示器，这给我们的系统更新以及后期维护带来了很大难题，我们可以想象如果使用C/S架构，更新客户端一旦遇到问题很可能需要整个厂区各个显示器跑一遍。这让我们在更新和维护方面面临较大压力。我们采用的B/S架构风格能够解决这个难题，并充分考量可现在的相关技术成熟度，例如现在的html5完全能够实现以前客户端的功能，项目中我们使用了大量的前端缓存技术与websocket技术，能够满足数据显示屏实时性交互等需求。这种风格中页面和逻辑处理存储在web服务器上，维护和软件升级只要更新服务器端即可，及时生效，用户体验较好，例如界面上需要优化，改一下javascript脚本或者CSS文件就可以马上看到效果了。

项目于2020年12月完成验收，这1年内共经历了2次大批量新购RFID读写器与EPC打印机接入，这几次接入过程平稳顺利，其中协议解释器软件性能没有出现过问题，中间件的性能经过多次调优，满足EPC信息读取性能效率。除了厂区几次突发性的停电故障外，项目至今还未有重大的生产事故，项目组现在留1个开发人员和1个实施工程师在维护，系统的维护量是可控的，系统运行也比较稳定。

不足之处有两个方面，第一在架构设计的过程中我们忽略数据看板问题，由于是价格的问题，不全部采购工控机，个别看板是智能电视机使用浏览器的方式显示内容，电视机浏览器不支持java代码，只支持单纯的JavaScript，所以导致布匹经过读写器时，看板没有反应。第二在系统容灾方面还有待改善。针对第一种问题，我们通过技术研讨会将前期使用WebSocket（需要部分java代码）的交互方式改成Ajax的轮询（纯JavaScript代码）的方式。针对第二种问题我方采用了服务器冗余和心跳监测等策略，在一台服务暂停的情况下，另外一台服务接管，以增加可用性。